ee2004

Week3,4

testbench

wa2.s * | wa2_tb.s *

```
1   ##TESTBENCH
2   #TEST 1
3   IN R0 is 1
4   IN R1 is 2
5   OUT R2 is 3
6   >>;          Test 1 PASSED.
7   #TEST 2
8   IN R0 is 10
9   IN R1 is 200
10  OUT R2 is 210
11  >>;          Test 2 PASSED.
```

wa2.s * | wa2_tb.s *

```
1
2           ADD     R2,R1,R0
3
4
```



A branch lets you jump to another place in the program

```
        MOV     R1, #0x1000         int* p = (int*)0x1000;
START   LDR     R0, [R1]            while(1){
        ADD     R0, R0, #1              *p++;
        STR     R0, [R1]            }
        B       START
```

Branching *conditionally* allows you to build if/while/for/case/etc.

```
        MOV     R1, #0x1000         int* p = (int*)0x1000;
START   LDR     R0, [R1]            while(*p != 100){
        ADD     R0, R0, #1              *p++;
        STR     R0, [R1]            }
        CMP     R0, #100
        BNE     START
```

Write code to loop through the numbers 1-10, and compute their sum.

Example implementation in C:

```c
int sum = 0;
for(int i = 1; i <= 10; i++){
    sum = sum + i;
}
```

```
1   main
2           mov     R1,#0 ; R1 is i
3           mov     R0,#0 ; R0 is sum
4
5   loop
6           add     R0,R0,R1 ; sum = sum + i
7           add     R1,R1,#1 ; i++
8           cmp     R1,#10 ; i <= 10
9           ble     loop
10
11  halt
12          b       halt
```

```c
int x[10];
int sum = 0;

for (int i = 0; i < 10; i++)
    sum += x[i];
```

```
1   veri    DCD     1, 3, 5, 7, 9, 11, 13, 15, 17, 19
2
3   sum     DCD     0
4
5   main    mov     r0,#veri
6           mov     r1,#sum
7           ldr     r2,[r1]
8           mov     r3,#0 ; r3 is i
9
10  for     ldr     r4,[r0,r3,lsl #2]
11          add     r2,r2,r4
12          add     r3,r3,#1
13          cmp     r3,#10
14          blt     for
15          str     r2,[r1]
16
17  halt    b       halt
```

```
2           MOV     R3, #5
3           MOV     R4, #5
4
5
6           CMP     R3,R4     ; if (i >= j)
7           BLT     else
8           SUB     R0,R3,R4  ; x = i - j;
9           B       endif     ; else
10  else    SUB     R0,R4,R3  ; x = j - i;
11  endif
12
13          CMP     R3,R4     ; x = (i >= j) ? i - j : j - i;
14          SUBGE   R0,R3,R4
15          SUBLT   R0,R4,R3
```

```
2   main
3           mov     r0, #35
4           mov     r1, #55
5           adr     r5, 0x300
6           bl      GCD
7
8   GCD
9           cmp     r0,r1
10          beq     halt
11          subgt   r0,r0,r1
12          sublt   r1,r1,r0
13          b       GCD
14
15  halt
16          str     r0, [r5]
17
```

```
int gcd(a,b) {
    while (a != b) {
        if (a > b) {
            a = a - b;
        } else {
            b = b - a;
        }
    }
    return a;
}

int x = 35;
int y = 55;
int z;

z = gcd(x, y);
```

Creating an Assembly Project in STM32CubeIDE

- We provide a template assembly code using the mentioned directives in Listing 15.1. To note here, this structure is specific to STM32CubeIDE. There may be extra directives or a different format for other development environments. Therefore, the reader should check them before preparing his or her assembly code in other IDEs.

- **.text** assembles the following code into text (executable code) section.
- **.align** defines the length of instructions to be processed in terms of bytes.
- **.global** defines one or more global symbols.
- **.end** identifies end of the code.

Listing 15.1 Template assembly code

```
.text
.align 2
.global main

main:

// Assembly Code Goes Here

END: B END
  .end
```

Listing 15.1 Template assembly code

```
 .text
 .align 2
 .global main

 main:

 // Assembly Code Goes Here

END: B END
  .end
```

- The first directive .text indicates that the following lines correspond to assembly code. Hence, they should be placed in the code section of the flash memory.
- The second directive .align defines the length of instructions to be processed in terms of bytes. If the instructions to be executed have 16-bit length, then we should use .align 2 as in our case.
- The third directive .global defines the global variable main which indicates the starting address of our code. Finally, the directive .end indicates the end of the assembly code.

Creating an Assembly Project in STM32CubeIDE

- There is no direct method to create an assembly project in STM32CubeIDE.
- Therefore, we should first follow the steps to form an empty C project.
- To create the debug file (with extension ".debug"), we should debug our (empty) C code for the first time.
- As the mentioned file is created, we should delete the "main.c" file from the "Src" folder of the project.
- In fact, we can delete all files with extension ".c" from the project's "Src" folder.
- Then, we should add a new file with extension ".s" to this folder.
- To do so, right-click on the "Src" folder, and press "New" and "File."
- In the opening window, enter the "File Name" such as "main.s" and click "Finish."
- An empty file will be created under the "Src" folder.
- We can add our assembly code to this file. Here, we will use the sample codes:

```
.text
.align 2
.global main

Mem_Addr=0x20000020

main:

 MOVS R0, #0x09
 MOVS R1, #0x11
 MOVS R2, #10

 LDR R3, =Mem_Addr
 STR R2, [R3]
 LDR R0, [R3]

END: B END
  .end
```

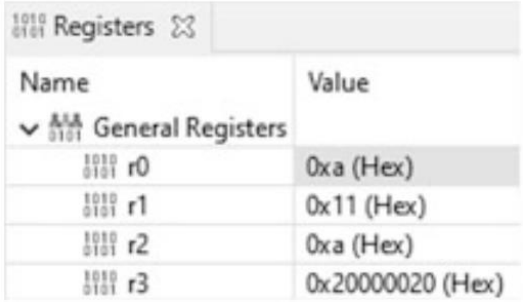
```
.text
.align 2
.global main

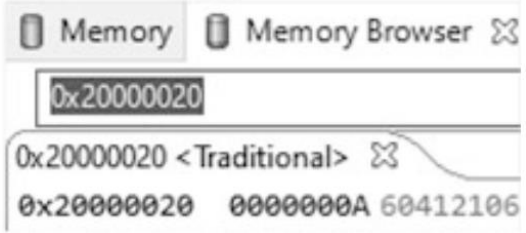
main:

 MOVS R0, #0x12
 MOVS R1, #0x2
 MOVS R2, #0x4
 ADD R3, R0, R1

END: B END
  .end
```

- As we compile the code, we can execute it either by the “step into” option in STM32CubeIDE or directly by the “Resume” command. We can observe output of the code; the registers have been modified according to our assembly code.
- We can apply operations on the memory of the STM32F4 microcontroller by assembly language. In the second code, we modify and reach content of the memory address 0x20000020 by the LDR and STR instructions. As we compile and run the code, we can observe its output in the “Registers” and “Memory Browser” Windows.





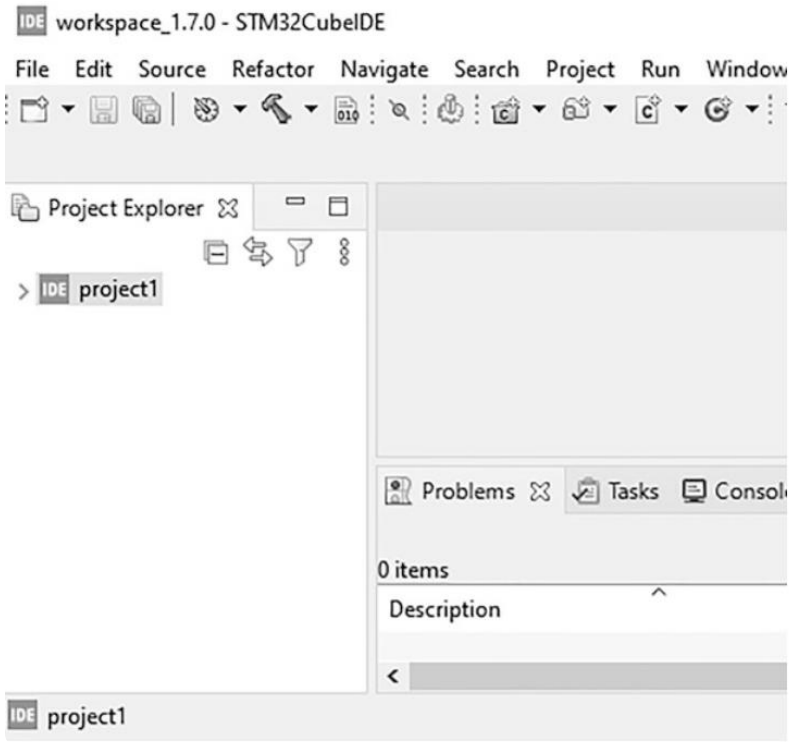
Lab1 solution

```
1
2
3           MOV     R0, #TESTDATA
4           MOV     R3, #8
5
6   loop    LDRB    R1, [R0]
7           ADD     R0, R0, #1 ; adres
8           ADD     R2, R2, R1 ; sonuç
9           SUB     R3, R3, #1 ; indeks
10          CMP     R3, #0
11          BNE     loop
12
13          LSR     R4, R2, #3
14
15          ADR     R5, 0x300
16          STR     R4, [R5]
17
18
19  TESTDATA DCD    0x0F1F2F3F, 0x4F5F6F7F
```

The STM32CubeIDE Platform

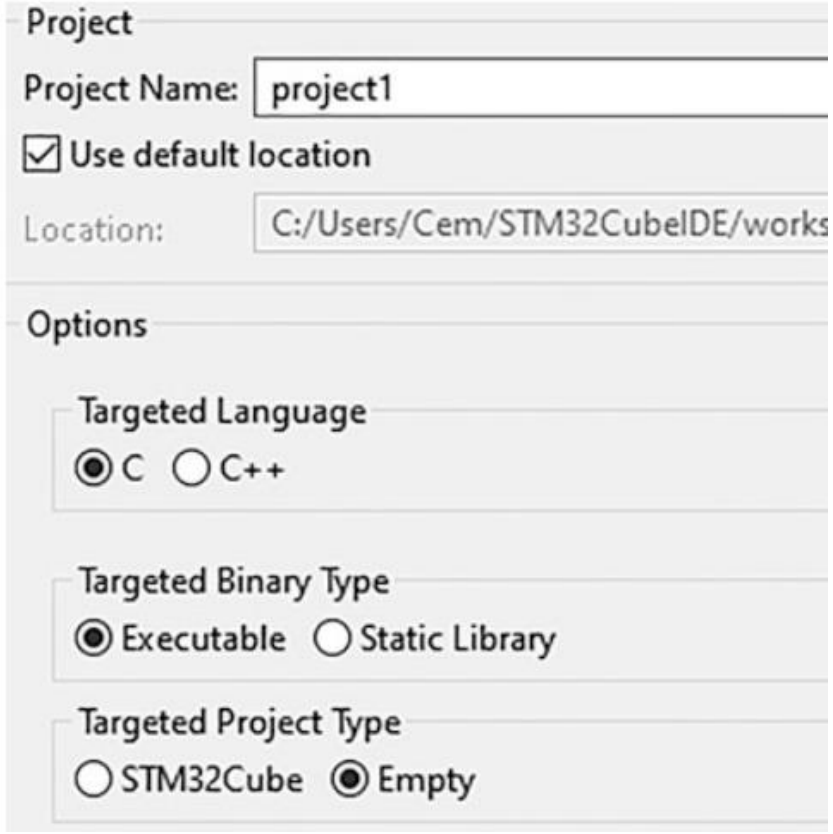
- We will benefit from the STM32CubeIDE platform to program the STM32F4 microcontroller via C language. This platform has several advantages. First, it is free. Second, it is specifically developed for microcontrollers by STMicroelectronics. Third, it is possible to reach and modify all properties of the microcontroller including its peripheral units using STM32CubeIDE.
- To get familiar with STM32CubeIDE, we will start with its installation. Afterward, we will explain how to create and manage a project. Here, we will not adjust peripheral units of the microcontroller. Instead, we will focus on fundamental properties of creating and executing a project in STM32CubeIDE. As we master this step, we will see how the peripheral units can be modified. To do so, we will benefit from STM32CubeMX which will be of great use in managing microcontroller hardware.
- STM32CubeIDE can be downloaded from the web site https://www.st.com/en/ development-tools/stm32cubeide.html after registering or logging in.

- There exists the “Project Explorer” panel on the left side of the welcome window. The user can create a new project through it.
- As a new project is created, this panel summarizes all its properties. Code files will be opened as they are added to the project at the center of STM32CubeIDE.
- Hence, the reader will modify his or her code in this section. There will be other sections which summarize hardware properties of the microcontroller on the right side of the panel.
- The reader can choose available perspectives such as “C/C++,” (default) “Debug,” and “Device Configuration Tool” in STM32CubeIDE.
- There are also grouped buttons on the top of the panel. These will be of use in debugging and modifying the code. There are tabs which will summarize all build and debug operations at the bottom of the panel. We will explain all these properties in the following sections as we focus on parts of a project.

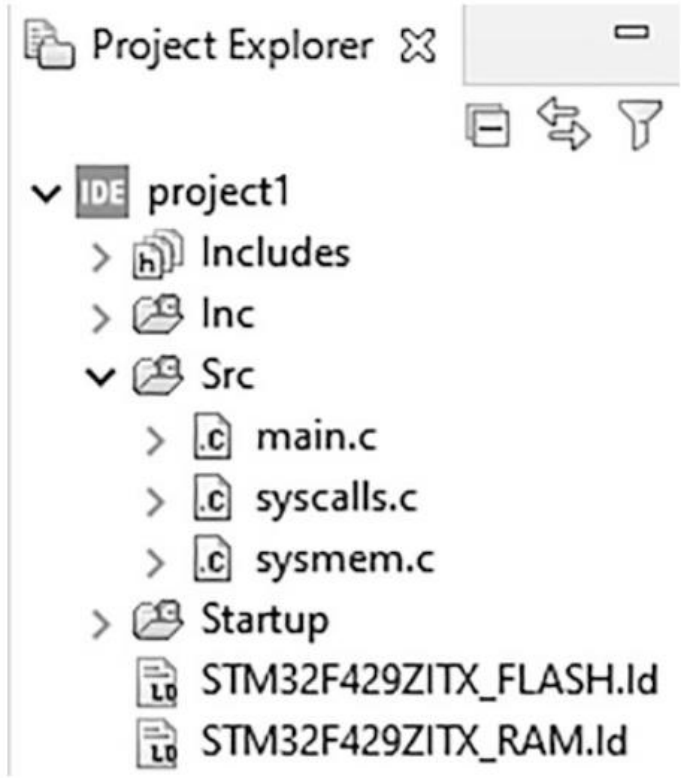


Creating a New Project

- A project typically contains source, header, and includes files. STM32CubeIDE generates an executable file from these to be embedded to the microcontroller. This section is about creating a C project. We have two options here. The first one does not deal with peripheral and pin properties of the microcontroller. The second one deals with these. We will focus on the first option in this section to get familiar with the basics of STM32CubeIDE.
- To create a new project, click “File,” “New,” and “STM32 Project.” In the opening window, select the hardware platform to be used in the project. For our case, we should choose “L073rz nucleo” from the “board selector” tab and press “Next.” Afterward, the “Project Setup” window opens as in Fig.



- We should give a name to our project in this window. For our case, we set it as “project1.”
- We should set the location of the project which may be done by the option “use default location.”
- We should set the “Targeted Language” as C.
- We should set the “Targeted Binary Type” as “Executable.”
- The final selection is titled “Targeted Project Type.” Here, we have two options as “STM32Cube” and “Empty.” The first option allows modifying the peripheral units.
- We select the “Empty” option to generate the project as for now. As we press the “Finish” button, a new project will be generated which can be observed under the “Project Explorer” tab as in Fig.



Listing 3.1 The first C code for the STM32F4 microcontroller

```
int a = 1;
int b = 2;
int c;

int main(void)
{
int d;


c = a + b;
d = c;

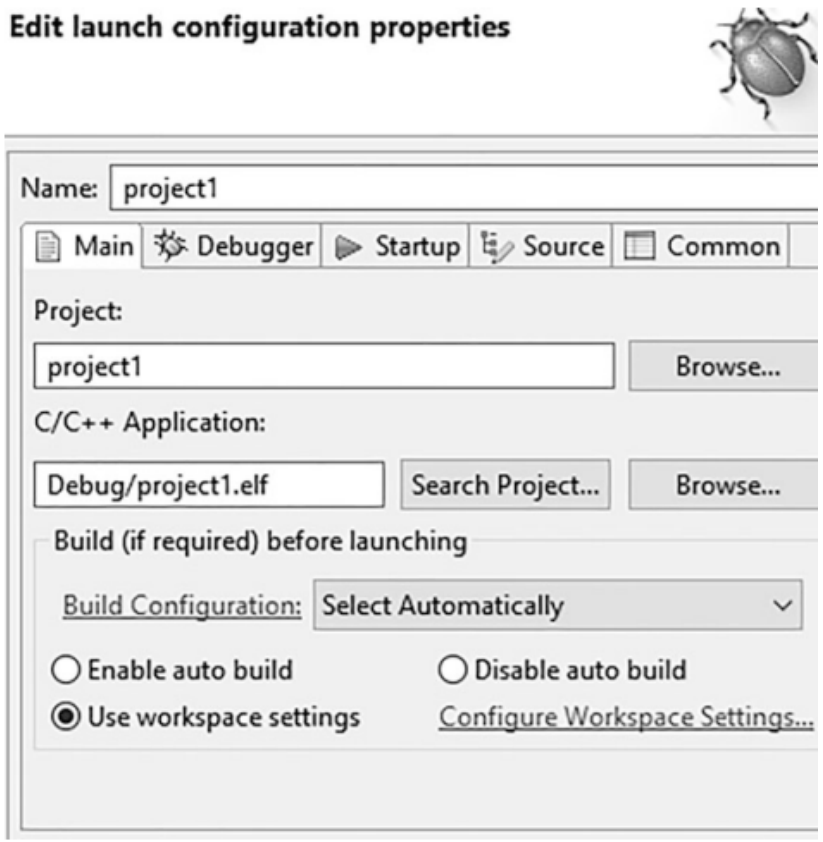
while (1);
}
```

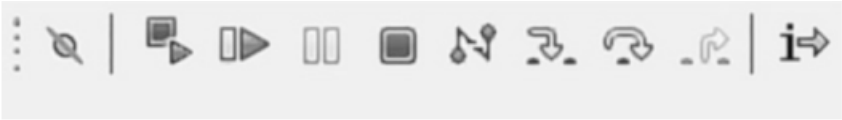
- The only step left is adding our C code to the “main.c” file generated under the “Src” folder. In this section, we will use the code given in Listing 3.1 as an example. To do so, open the “main.c” file by double-clicking on it. Add your code to this file. Then, save it by clicking the “Save” button on the upper left corner of the menu.

Building, Debugging, and Executing the Project

- As we create our project, the next step is its execution. To do so, we will first consider the building and debugging steps. Afterward, we will focus on the execution step.
- The first step in executing the code on an embedded platform is building and debugging it. There is a button with hammer shape on STM32CubeIDE horizontal toolbar as “Build”. As the user presses the right bottom arrow there, two options emerge as “Debug” and “Release”.
- The first option builds the project with the necessary debug information. In this default option, no optimization is done on the built output.
- The second option is specific to deploying the built output to the microcontroller. Hence, the main focus here is optimizing output of the project and minimizing its size. Running steps in both options (such as warnings and errors) can be observed in the “CDT Build Console” window. If there is an error in a code line, it is indicated by a red cross.

- The second step in executing the code is debugging and loading it to the target device. There is a button on STM32CubeIDE's horizontal toolbar as "Debug." It can also be reached from the "Run" tab. As we press this button, a new window appears asking for the "Edit Configuration" as in Fig. 3.4. Within this window, we can make all adjustments related to the debugging process. The default settings should be kept as they are in this window unless otherwise stated.
- As we press "OK" in the "Edit Configuration" window, the code is debugged and downloaded to the microcontroller. Afterward, the STM32CubeIDE interface turns to the "Debug" perspective.





- As the debugged C code is downloaded to the microcontroller, the next step is its execution. At this stage, the code will be executed till the beginning of the main function. The microcontroller waits for further command. Buttons for program execution are located on the toolbar as in Fig. 3.5.
- The name of each button can be observed by moving the cursor over it. These buttons and their function are explained briefly in the below list.
- **Skip all breakpoints**: Disable all previously set breakpoints. We will talk about what a breakpoint is in the following paragraphs.
- **Terminate and relaunch**: If a modification is done in the program while in debug mode, this button rebuilds and debugs it.
- **Resume**: Resumes execution of the code from the last executed location. Execution of the code continues until a breakpoint when this button is clicked.
- **Suspend**: Halts execution of the code. All windows used to observe software and hardware are updated with recent data.
- **Terminate**: Ends execution of the code. The debug session is also ended by this command.
- **Disconnect**: Disconnect from the target device.
- **Step into**: Executes the next line of code. If this line calls a function, the compiler executes the next line in it. Then, it stops.
- **Step over**: Executes the next line of code. If this line calls a function, the compiler executes the function completely. Then, it stops.
- **Step return**: Completes execution of the function and exits the code.
- **Instruction stepping mode**: The C code is executed based on its generated assembly code within the step into, step over, and step return commands

- Observing variables, registers, or memory is important while developing the project. Code execution should halt in order to perform this operation. A breakpoint should be added to stop execution of the code at a specific code line. To do so, left-click on the desired code line and select “Add Breakpoint. . . ” from the popup window. A new window appears asking for the breakpoint type. Here, we can select the “Regular” type. As the breakpoint is added, a blue circle will appear by the code line to indicate that there is a breakpoint there. The inserted breakpoint can be deleted by double-clicking on it.
- The “Expressions” window (given in Fig) can be used to observe selected variables. In order to add a variable to this window, select the variable to be observed and right-click on it. Select the “Add Watch Expression” option in the opening list. Then, click “OK.” The reader can also double-click on the “Add new expression” button in the “Expressions” window and enter the name of the variable to the opened box

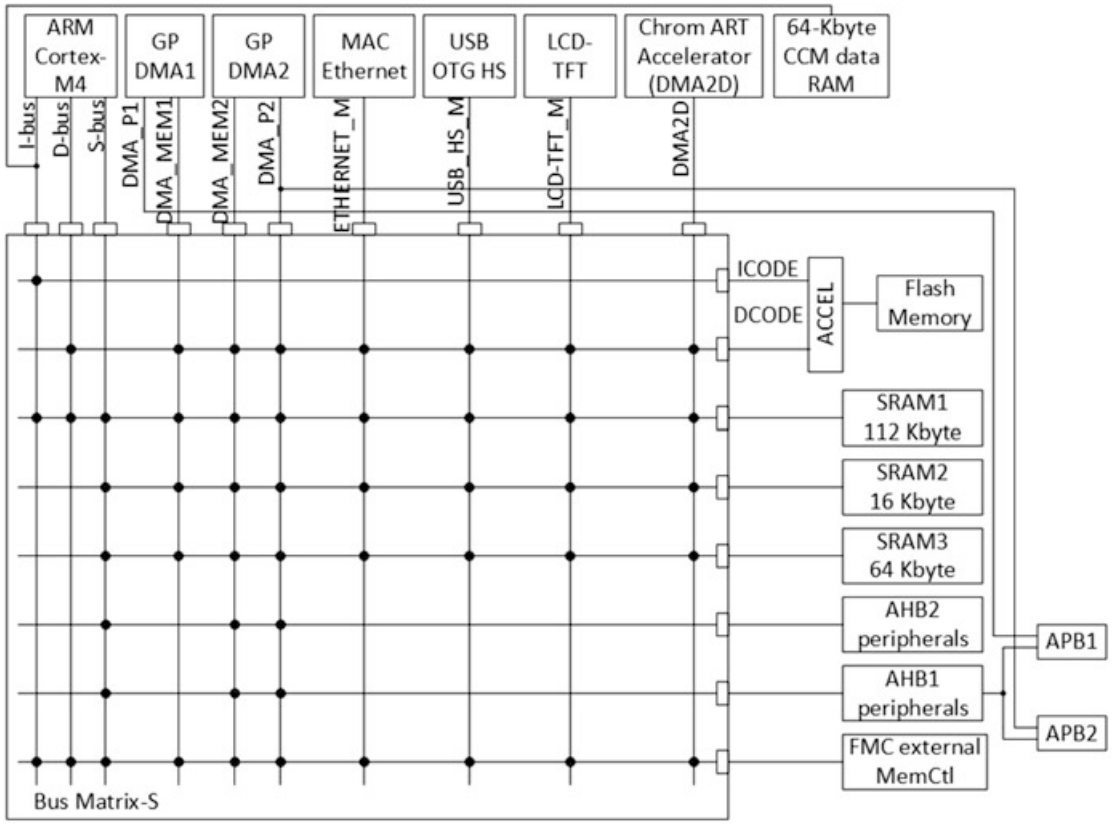
Expression	Type	Value
(x)= a	int	1
(x)= b	int	2
(x)= c	int	0
(x)= d	int	0

- We can define a variable either as local or global in C language. As the name implies, the global variable is available to all code sections. The local variable is only available to the function it is defined in. We will explore these variable types in detail. Although local and global variables can be observed in the “Expressions” window, local variables can also be observed in the “Variables” window. Here, all local variables are automatically added to the mentioned window.
- There is also “Live Expressions” window which can be enabled from the “Window/Show View” list besides other options. In this window, variables are updated in real time while executing the code. Hence, value of a variable can be observed without stopping the code.

Bus Architecture

- A bus is composed of parallel wires in which data and instructions are transferred between microcontroller units. Arm® microcontrollers use advanced microcontroller bus architecture (AMBA) for this purpose. AMBA controls all data transfer between microcontroller units. Here, the unit which initiates data transfer is called master. The unit which responds to master is called slave.
- There are two different bus structures in AMBA. These are called advanced high-performance bus (AHB) and advanced peripheral bus (APB). AHB supports high-speed and high bandwidth operations. Therefore, it is generally used by CPU, DMA modules, and external and internal memory units. AHB supports pipelined operations and burst transfer. Burst transfer is used to realize multiple data transactions at once. This is especially useful for communicating with slave units with high initial access latency. APB supports low bandwidth operations. This leads to low power consumption. Communication between most peripheral units is performed via this interface.

- The STM32F4 microcontroller has a multilayer bus matrix as in Fig. 9.1. This bus matrix allows connecting several master and slave units. To be more specific, the STM32F4 microcontroller has 10 master and 8 slave units.
- As can be seen in Fig. 9.1, there are three bus types connecting to the bus matrix as instruction bus (I-bus), data bus (D-bus), and system bus (S-bus). The instruction bus is used by the CPU to fetch instructions from flash memory, SRAM, or external memory (wherever the code is running from). The prefetch operation in the adaptive real-time accelerator, to be introduced in Sect. 9.1.4, is also performed via the instruction bus.
- The CPU controls code or data in memory locations by the data bus. The system bus allows the CPU to reach peripheral units or SRAM. Besides, DMA buses connect the related DMA master devices to desired peripheral unit and memory blocks. Hence, DMA can be used for providing high-speed data transfer between peripheral units to memory, memory to peripheral units, and memory to memory without the CPU intervention. We will evaluate these operations in detail in Sect. 9.4.



Memory in General

- The memory in the STM32F4 microcontroller is based on Harvard architecture. Hence, data and instructions are transferred through their own dedicated data and instruction bus.
- The number of possible memory locations is determined by the address bus size. There are $2k$ addressable memory locations for k address bits. AHB and APB of the STM32F4 microcontroller have 32 bit address bus. Hence, there are a total of 4 GByte possible usable address locations. Besides, each address location can keep 1 byte data as will be explained next. Hence, the maximum data size that can be stored in memory is 4 GByte.

- Memory is composed of data storage blocks. Each block has its own address value and content. As the name implies, address of a memory block indicates its unique location in memory. The content is the value stored in that address.
- The minimum memory block size for the STM32F4 microcontroller is 1 byte. In other words, the minimum addressable memory block can have size of 1 byte. Hence, these memory blocks are called byte-addressable.
- For RAM, the addressable memory size can be 1 byte, 2 bytes (half-word), and 4 bytes (word).
- For the flash memory, the addressable memory size can be 1 byte, 2 bytes (half-word), and 4 bytes (word) for the writing operation.
- We can also have 8 byte (double word) addressable memory size in flash memory for reading operation. Since the minimum addressable memory location is 1 byte, half-word and word data are stored in the memory in little endian format.

RAM

- SRAM of the STM32F4 microcontroller is divided into different blocks as tabulated in Table 9.1. In this table, CCM stands for core coupled memory. Backup SRAM can be used as internal EEPROM in standby mode. Read and write operations to SRAM are performed at any clock speed without delay.
- As can be seen in Table 9.1, SRAM1, SRAM2, and SRAM3 form a total of 192 kB SRAM used by global and static variables, heap, and stack.

Table 9.1 SRAM blocks and their properties

Name	Size (kB)	Start address	Accessed by
SRAM1	112	`0x20000000`	All AHB master units
SRAM2	16	`0x2001C000`	All AHB master units except D-bus and I-bus
SRAM3	64	`0x20020000`	All AHB master units except D-bus and I-bus
CCM data RAM	64	`0x10000000`	D-bus
Backup SRAM	4	`0x40024000`	All AHB master units

- Heap is a specific RAM region in which dynamic memory allocation can be performed.
- Stack is the specific memory region initially located at the end of usable SRAM. Here local variables and temporary data are stored during code execution. Data is moved to and removed from stack in last-in first-out (LIFO) order. This means last data moved to stack is removed first.
- Stack memory allocation and deallocation is done automatically in C and C++ languages. However, stack should be used carefully to prevent stack overflow. This overflow occurs when variables added to the stack exceed its free area during code execution. For such cases, either existing variables or critical code segments in nearby RAM regions are overwritten. This may cause serious problems.

Flash Memory

- The STM32F4 microcontroller has 2 MB flash memory divided into two banks, each having 1 MB space. Each bank is further divided into 12 sectors. Hence, sectors from 0 to 11 are in bank 1 and sectors from 12 to 23 are in bank 2. For each bank, first four sectors are 16 kB, fifth sector is 64 kB, and remaining sectors are 128 kB. Starting address of flash memory (beginning of sector 0) is 0x08000000 and end address (end of sector 23) is 0x081FFFFF. Flash memory erasing operation can be done at sector level, bank level, or whole (mass flash erase).
- The CPU has higher clock speed compared to flash memory. Therefore, when CPU performs an operation connected to flash memory, it should wait for the end of operation. This is called wait state.

- The CPU has higher clock speed compared to flash memory. Therefore, when CPU performs an operation connected to flash memory, it should wait for the end of operation. This is called wait state. To be more specific, when reading data or code from flash memory, there is a relation between HCLK frequency, supply voltage level, and number of flash wait states. If the supply voltage is kept constant, then the wait state increases with respect to the HCLK frequency. As an example, when the HCLK frequency is in the range 0–30 MHz, the wait state will be zero clock cycle. When the HCLK frequency is in the range 150–180 MHz, the wait state becomes five clock cycles. Hence, increasing the clock speed does not directly increase performance. To overcome this problem, the STM32F4 microcontroller has the adaptive real-time (ART) accelerator unit. Normally, 128-bit read operations can be performed sequentially and flash is stalled during number of wait states between each read. The ART accelerator uses an instruction prefetch queue and branch cache. Hence, latency between successive reads may be prevented. As a result, 0 wait state code execution from flash memory may be achieved at CPU clock frequencies up to 180 MHz

- The STM32F4 microcontroller also has other memory regions besides its 2 MB flash memory. These are tabulated in Table 9.2. In this table, OTP stands for onetime programmable. System memory region is used to boot the device in system memory boot mode. As the name implies, OTP bytes can be programmed only once and cannot be rewritten again. Option bytes region is used to configure read and write protection, brown-out reset (BOR) level, watchdog module, and dual bank boot mode. When the device is in standby or stop mode, it can also be used to adjust the reset operation.

Name	Size (bytes)	Start address
System memory	32,768	`0x1FFF0000`
OTP bytes	528	`0x1FFF7800`
Option bytes, bank 1	16	`0x1FFFC000`
Option bytes, bank 2	16	`0x1FFEC000`

Memory Management in C and C++ Languages

- We will consider memory management in C and C++ languages in this section. Since both languages share common properties, we handle them together. We will start memory management with RAM and flash memory modification. Then, we will focus on pointers and their usage for memory management. Afterward, we will handle local and global variables in C and C++ languages.

Lab2 solution

Memory filling

```
1   Loc1    EQU     0x20000400
2   Loc2    EQU     0x30002000
3
4
5   memPrep LDR     R0, =Loc1
6           LDR     R1, =0xF0F3F7F1
7           STR     R1, [R0]
8           ADD     R0, R0, #4
9           LDR     R1, =0xFaFbFcFd
10          STR     R1, [R0]
11          ADD     R0, R0, #4
12          LDR     R1, =0xF1FaFeF3
13          STR     R1, [R0]
14          ADD     R0, R0, #4
15          LDR     R1, =0xFfFcF9F1
16          STR     R1, [R0]
17
```

```
1   Loc1    EQU     0x20000400
2   Loc2    EQU     0x30002000
3
4
5   memPrep LDR     R0, =Loc1
6           LDR     R1, =0xF0F3F7F1
7           STR     R1, [R0]
8           ADD     R0, R0, #4
9           LDR     R1, =0xFaFbFcFd
10          STR     R1, [R0]
```

```
main    MOV     R1, #0x0 ; summation
        MOV     R2, #0x0
        MOV     R3, #0x1 ; mask
        LDR     R0, =Loc1
        LDR     R2, [R0]
        MOV     R7, #2 ; dış indeks


loop0   LDR     R2, [R0]
        MOV     R6, #4; indeks

loop1   AND     R5, R3, R2
        ADD     R1, R1, R5
        LSR     R2,R2, #8
        SUB     R6,R6, #1
        CMP     R6, #0
        BNE     loop1

        ADD     R0, R0,#4
        SUB     R7, R7, #1
        CMP     R7, #0
        BNE     loop0

result  LDR     R0, =Loc2
        STR     R1, [R0]; tot num of odd numbers
```

```
main    MOV     R1, #0x0 ; summation
        MOV     R2, #0x0
        MOV     R3, #0x1 ; mask
        LDR     R0, =Loc1
        LDR     R2, [R0]
        MOV     R7, #4 ; dış indeks


loop0   LDR     R2, [R0]
        MOV     R6, #4; indeks

loop1   AND     R5, R3, R2
        ADD     R1, R1, R5
        LSR     R2,R2, #8
        SUB     R6,R6, #1
        CMP     R6, #0
        BNE     loop1

        ADD     R0, R0,#4
        SUB     R7, R7, #1
        CMP     R7, #0
        BNE     loop0

result  LDR     R0, =Loc2
        STR     R1, [R0]; tot num of odd numbers
```